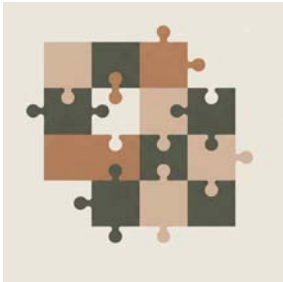


B 3.2.4 Composition and aggregation (HL Only)



B3.2.4 Explain the role of composition and aggregation in class relationships. (HL Only)

This section explores composition and aggregation in OOP, focusing on how to design objects using smaller components. It differentiates between aggregation (subcomponents can exist independently) and composition (subcomponents tightly coupled, cannot exist outside the aggregating class).

Learning Objectives

- Students will be able to explain the concepts of composition and aggregation in object-oriented programming.
- Students will be able to design Java/Python classes that demonstrate "has-a" relationships using composition.
- Students will be able to design Java/Python classes that demonstrate "has-a" relationships using aggregation.
- Students will be able to differentiate between composition and aggregation based on the dependency and lifecycle of the related objects.

Relevant ATL Skills:

Thinking Skills:

- **Critical thinking:** Evaluating the suitability of composition vs. aggregation for different design scenarios.
- **Problem-solving:** Designing class structures to model real-world relationships effectively.
- **Abstraction:** Representing complex relationships through simplified class structures.

Communication Skills:

- Using appropriate terminology to explain composition and aggregation.
- Communicating design choices and justifications.

Self-Management Skills:

- Organisation: Structuring code logically using classes and relationships.

Lesson Resources

Presentation Java

[Link to presentation](#)

Presentation Python

[Link to presentation](#)

eBook

[B 3.2.4 Composition and aggregation Java \(HL Only\)](#)

[B 3.2.4 Composition and aggregation Python \(HL Only\)](#)

Other resources

[B 3.2.4 Worksheet Java](#)

[B 3.2.4 Worksheet Python](#)

[B 3.2.4 Worksheet Answers](#)

Lesson Planning

Activity	Content Covered	Slide(s)
Introduction & Objectives	Review of OOP fundamentals, introduction to "has-a" relationships, lesson objectives, and relevant ATL skills.	1-2
Concept Exploration: Composition	Definition of composition, characteristics (tight coupling, dependent lifecycle), example (Car and Engine).	3-5

Activity	Content Covered	Slide(s)
Concept Exploration: Aggregation	Definition of aggregation, characteristics (loose coupling, independent lifecycle), xample (Library and Book).	6-8
Comparing Composition & Aggregation	Key differences summarised, scenarios for choosing each relationship type.	9-10
Activity: Design Challenge	Students work individually or in pairs on a design problem requiring the use of composition and/or aggregation (e.g., University and Department/Student).	11
Class Discussion & Examples	Students share their design choices and reasoning, teacher provides feedback and reinforces concepts.	-
Wrap-up & Worksheet Intro	Review of key concepts, introduction to the worksheet for independent practice.	12 (Worksheet Introduction)

Teacher Notes

Teacher Notes (Java):

- **Potential Challenge:** Students might confuse the syntax of creating objects within another class with composition itself. Emphasise that the *conceptual* dependency and lifecycle are the defining factors, not just the location of the `new` keyword.
- **Lifecycle and Garbage Collection:** Explain how in composition, when the "whole" object in Java becomes eligible for garbage collection, the "part" object (if only referenced by the whole) will also be garbage collected, illustrating the dependent lifecycle. In aggregation, the aggregated objects can have other references and persist independently.
- **Navigability:** Briefly touch upon navigability in UML diagrams (the arrow direction) to indicate which class knows about the other. While not the core of composition/aggregation, it's a related concept that might arise.
- **Example Variety:** Use a diverse range of Java-centric examples to illustrate the concepts. For instance, a `House` and its `Bathroom` (composition), a `Course` and the `Instructor` teaching it (aggregation - the instructor exists independently).
- **Immutability and Composition:** For more advanced discussions, you could briefly mention how using final fields for composed objects can reinforce immutability and the strong relationship.

- **Justification is Key:** As with Python, encourage students to justify their choices on the worksheet clearly, focusing on the "could the part exist meaningfully without the whole?" and "is the part exclusively owned by the whole?" questions.

Teacher Notes (Python):

- **Potential Challenge:** Students might struggle to grasp the subtle difference in the lifecycle dependency. Emphasise that in composition, if the "whole" object is destroyed, the "part" object typically goes with it (or is exclusively owned by it). In aggregation, the "part" can exist independently.
- **Clarifying Creation:** Point out that while in the composition example, the **Engine** is created *within* the **Car**, composition isn't solely about where the object is instantiated in the code. The key is the conceptual dependency. For instance, a **Car** might be constructed with an **Engine** object passed to it, but if that specific **Engine** is considered integral and not meant to be shared or exist outside that **Car**, it's still a compositional relationship.
- **Python Specifics:** In Python, there isn't a strict language-level enforcement of composition vs. aggregation in the same way some other languages might have through features like inner classes (which can imply stronger coupling). The distinction in Python is largely conceptual and based on how the objects are managed and their intended relationships. Encourage students to think about the design intent.
- **Analogy Reinforcement:** Use various real-world analogies to help solidify the concepts. For example, a body and its organs (composition), a sports team and its players (aggregation).
- **Addressing Ambiguity:** Some relationships can be argued as either composition or aggregation depending on the specific context and design choices. Encourage students to justify their reasoning clearly. For example, a "Book and its Pages" could be seen as composition (a book without pages isn't really a book), but individual pages could theoretically exist outside the specific binding. Discussing such ambiguities can lead to deeper understanding.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.
